Database Choices — Design Rationale & Architecture

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Design Philosophy — Polyglot Persistence
2. Database Selection Matrix
3. PostgreSQL — Primary Transactional Store
4. MongoDB — Audit & Event Logs
5. Redis — Cache, Locks & Session Store
6. Elasticsearch — Search & Analytics
7. Alternatives Evaluated & Rejected
8. Database-Per-Service Ownership Model
9. Data Consistency Across Stores
10. Operational Considerations
11. Cost Analysis

---

1. Design Philosophy — Polyglot Persistence

We use **four different data stores**, each chosen for a specific access pattern. This is polyglot persistence — the principle that no single database is optimal for all workloads. In a banking system, the penalty for choosing the wrong database is measured in downtime, data loss, or regulatory non-compliance.

```
┌──────────────────────────────────────────────────────────────────────┐
│                     DATA LAYER — POLYGLOT PERSISTENCE                 │
│                                                                       │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌─────────┐ │
│   │  PostgreSQL  │  │   MongoDB    │  │    Redis     │  │   ES    │ │
│   │              │  │              │  │              │  │         │ │
│   │ OLTP         │  │ Write-heavy  │  │ Sub-ms reads │  │ Search  │ │
│   │ ACID txns    │  │ Append-only  │  │ Ephemeral    │  │ Aggs    │ │
│   │ Relational   │  │ Schema-free  │  │ In-memory    │  │ Text    │ │
│   │ Consistency  │  │ High volume  │  │ Distributed  │  │ Fuzzy   │ │
│   └──────────────┘  └──────────────┘  └──────────────┘  └─────────┘ │
│                                                                       │
│   USE FOR:          USE FOR:          USE FOR:          USE FOR:      │
│   • Accounts        • Audit logs      • Sessions        • Txn search  │
│   • Transactions    • Login history   • OTP cache       • Full-text   │
│   • Users           • API access logs • Balances (cache) • Analytics  │
│   • Loans           • Dispute trails  • Rate limits      • Facets     │
│   • Sagas           • Notifications   • Distributed locks              │
│   • Schedules       • Fraud alerts    • Idempotency keys               │
└──────────────────────────────────────────────────────────────────────┘
```

2. Database Selection Matrix

Criteria	PostgreSQL	MongoDB	Redis	Elasticsearch
Data model	Relational (tables, rows, foreign keys)	Document (JSON/BSON)	Key-value, sorted sets, hashes	Inverted index (documents)
Consistency	Strong (ACID, serializable)	Eventual (tunable)	Eventual (single-key atomic)	Eventual
Latency	1-50ms (indexed queries)	1-20ms (document reads)	< 1ms	10-200ms
Write pattern	Moderate writes with constraints	High-volume append-only	Fast ephemeral writes	Batch indexing
Scale strategy	Vertical + read replicas	Horizontal (sharding)	Cluster (hash slots)	Horizontal (shards)
Durability	WAL + sync replication	Journaled, replica set	Optional (RDB/AOF)	Replica shards
Query power	SQL, joins, window functions, CTEs	Aggregation pipeline	Simple key lookups	Full-text, fuzzy, aggregations
Banking fit	Financial transactions, regulatory compliance	Immutable logs, flexible schema events	Sub-ms caching, locking	User-facing search

3. PostgreSQL — Primary Transactional Store

3.1 Why PostgreSQL?

Requirement	How PostgreSQL Meets It
ACID transactions	Full ACID with serializable isolation. A banking transfer (debit + credit) either both succeed or both roll back. No partial states.
Referential integrity	Foreign keys enforce: every transaction belongs to a valid account, every loan links to a user. The database enforces business invariants the application cannot bypass.
Optimistic locking	`@Version` column + `UPDATE ... WHERE version = ?` prevents lost updates on concurrent balance modifications.
Pessimistic locking	`SELECT ... FOR UPDATE` row-level locks for high-contention accounts (merchant accounts).
CHECK constraints	`CHECK (balance ≥ 0)` — the database itself prevents negative balances as a last line of defense.
Range partitioning	`PARTITION BY RANGE (created_at)` on the transactions table. Monthly partitions allow partition pruning, independent vacuuming, and eventual archival.
Full-text search (fallback)	`GIN` index on `to_tsvector('english', description)` provides degraded search when Elasticsearch is down.
Regulatory compliance	Deterministic `NUMERIC(18,2)` type for money — no floating-point precision loss. Critical for audit and reconciliation.
Mature ecosystem	Flyway migrations, PgBouncer connection pooling, pg_partman automated partitioning, pgAudit for compliance logging.

3.2 Why Not MySQL?

Feature	PostgreSQL	MySQL (InnoDB)
Partitioning	Native range, list, hash partitioning with partition pruning	Supported but more limited (no foreign keys on partitioned tables until 8.0.21)
JSON support	Full `JSONB` type with indexing	JSON type exists but less performant for queries
Partial indexes	`CREATE INDEX ... WHERE status ≠ 'SUCCESS'`	Not supported
CTEs (WITH queries)	Full support, recursive CTEs	Supported since 8.0, but historically weaker
Concurrent index creation	`CREATE INDEX CONCURRENTLY` — zero downtime	`ALTER TABLE ... ADD INDEX` locks the table in many scenarios
Row-level security	Native `CREATE POLICY`	Not available
Extension ecosystem	PostGIS, pg_partman, pgvector, pgAudit	More limited

PostgreSQL is the industry standard for financial-grade RDBMS workloads. MySQL is viable but PostgreSQL's richer feature set (partial indexes, table partitioning, concurrent DDL, row-level security) aligns better with banking requirements.

3.3 Why Not Oracle or SQL Server?

- **Licensing cost:** PostgreSQL is open source. Oracle/SQL Server licensing at scale (multi-AZ, read replicas) costs 10-50x more.
- **Cloud portability:** PostgreSQL runs on AWS RDS, Aurora, GCP Cloud SQL, Azure — no vendor lock-in. Oracle on RDS has limitations.
- **Feature parity:** PostgreSQL 16+ matches Oracle on partitioning, JSON, window functions, and CTEs.
- **Operational:** Larger open-source community, better integration with Kubernetes (CloudNativePG), and more deployment options.

Oracle may be justified for legacy integrations with CBS (Core Banking System), but for a greenfield microservices architecture, PostgreSQL is the superior choice.

3.4 PostgreSQL Configuration for Banking

```yaml
# Key PostgreSQL tuning parameters (production)

# Memory
shared_buffers: 8GB              # 25% of server RAM (32GB server)
effective_cache_size: 24GB       # 75% of server RAM
work_mem: 64MB                   # Per-sort/hash operation
maintenance_work_mem: 1GB        # For VACUUM, CREATE INDEX

# WAL & Durability
wal_level: replica               # Enables streaming replication
synchronous_commit: on           # Never lose committed transactions
max_wal_senders: 10
wal_keep_size: 2GB

# Connection Pooling (via PgBouncer, not PG native)
max_connections: 200             # PgBouncer multiplexes to fewer PG connections

# Partitioning
enable_partition_pruning: on     # Critical for transaction table performance

# Replication
primary_conninfo: 'host=pg-replica-1 port=5432'
hot_standby: on                  # Read replicas accept read queries
```

3.5 Deployment Topology

```
┌──────────────────────────────────────────────────────────────┐
│                    PostgreSQL Deployment                     │
│                                                              │
│   ┌──────────────────┐     sync repl     ┌──────────────────┐    │
│   │ Primary (AZ-1)   │ ────────────────► │ Standby (AZ-2)   │    │
│   │ (Read + Write)   │                   │ (Auto-failover)  │    │
│   └──────────────────┘                   └──────────────────┘    │
│            │                                                     │
│            │                                                     │
│            │ async repl                                          │
│            ▼                                                     │
│   ┌──────────────────┐     ┌──────────────────┐                  │
│   │ Read Replica 1   │     │ Read Replica 2   │                  │
│   │ (AZ-1)           │     │ (AZ-2)           │                  │
│   │                  │     │                  │                  │
│   │ Used by:         │     │ Used by:         │                  │
│   │ • Filter queries│     │ • Report Service│                  │
│   │ • BFF reads      │     │ • Analytics      │                  │
│   │ • Search fallback│     │ • Statement gen │                  │
│   └──────────────────┘     └──────────────────┘                  │
│                                                                  │
│  PgBouncer in front of every connection (transaction mode)       │
└──────────────────────────────────────────────────────────────┘
```

4. MongoDB — Audit & Event Logs

4.1 Why MongoDB for Audit?

Requirement	How MongoDB Meets It
Schema flexibility	Audit events from different services have different payloads. A login event looks nothing like a transfer event. MongoDB's document model handles heterogeneous schemas naturally without ALTER TABLE migrations.
Append-only, immutable	Audit logs are write-once, never updated or deleted (compliance). MongoDB's append-heavy write pattern is ideal. Capped collections can enforce immutability.
High write throughput	Thousands of audit events per second from all services. MongoDB sharding handles horizontal write scaling.
Flexible queries	Compliance officers query by: userId, action type, date range, IP address, resource type. MongoDB's secondary indexes and aggregation pipeline handle these ad-hoc queries.
TTL indexes	`db.auditLogs.createIndex({timestamp: 1}, {expireAfterSeconds: 31536000})` — automatic cleanup after retention period, no cron jobs.
No relational needs	Audit logs are self-contained documents — no joins needed. Each document has the full context (who, what, when, where, result).

4.2 Why Not PostgreSQL for Audit?

Issue	Detail
Schema rigidity	Each new audit event type would require ALTER TABLE or JSONB (which negates the relational model's benefits).
Write contention	High-volume inserts on the same table compete with transactional reads. Separate database is cleaner.
Index bloat	Billions of audit rows cause index bloat and slow VACUUM. PostgreSQL handles this but requires more operational tuning than MongoDB's LSM-tree approach.
Storage cost	MongoDB compresses documents (Snappy/Zstandard) better than PostgreSQL's TOAST for large text payloads.

4.3 Why Not Elasticsearch for Audit?

Elasticsearch is used for *search* but not as the audit log's primary store because:

- **Durability guarantees:** ES is not designed as a system of record. It can lose data during shard rebalancing or node failures. MongoDB's replica set with `w: majority` provides stronger durability.
- **Update/delete risk:** ES allows document updates and deletions. MongoDB collections can be configured as effectively immutable (deny `update` and `delete` operations at the role level).
- **Cost:** Storing billions of audit documents in ES is significantly more expensive than MongoDB due to ES's inverted index overhead.

We do index a subset of audit data into ES for the admin search portal, but MongoDB remains the source of truth.

4.4 MongoDB Document Schema

```
{
  "_id": ObjectId(" ... "),
  "auditId": "aud-123",
  "timestamp": ISODate("2026-04-15T10:30:00Z"),
  "actor": {
    "userId": "user-007",
    "role": "CUSTOMER",
    "ip": "203.0.113.45",
    "userAgent": "Mozilla/5.0 ... ",
    "deviceId": "device-abc"
  },
  "action": "TRANSFER_INITIATED",
  "resource": {
    "type": "TRANSACTION",
    "id": "txn-456"
  },
  "details": {
    "fromAccount": "ACC-001 (masked)",
    "toAccount": "ACC-002 (masked)",
    "amount": 5000.00,
    "channel": "WEB"
  },
  "result": "SUCCESS",
  "correlationId": "req-789",
  "traceId": "jaeger-trace-abc",
  "sourceService": "transaction-service"
}
```

4.5 MongoDB Deployment

```
MongoDB Atlas Replica Set (3 nodes, multi-AZ)

┌──────────┐    ┌──────────┐    ┌──────────┐
│ Primary  │    │ Secondary │    │ Secondary │
│ (AZ-1)   │──→│ (AZ-2)   │──→│ (AZ-3)   │
│          │    │          │    │          │
│ Writes   │    │ Reads    │    │ Reads    │
└──────────┘    └──────────┘    └──────────┘

Write concern: w:majority (2 of 3 nodes acknowledge)
Read preference: secondaryPreferred (for audit queries)
```

---

5. Redis — Cache, Locks & Session Store

5.1 Why Redis?

Requirement	How Redis Meets It
Sub-millisecond reads	In-memory store with < 1ms latency. Account balance cache, session validation, OTP verification — all need microsecond response.
Distributed locking	`SET key value NX EX 10` — atomic lock acquisition with auto-expiry. Used for preventing concurrent account modifications.
Session store	JWT sessions stored as key-value with TTL. `SET session:{token} {userId} EX 3600` for 1-hour sessions.
Atomic operations	`INCR`, `DECR`, `SET NX` — atomic primitives power rate limiting (`INCR + EXPIRE`), idempotency checks (`SET NX`), and real-time counters.
Data structures	Sorted sets for leaderboards, lists for recent transactions, hashes for structured session data. Beyond simple key-value.
Pub/Sub	Redis Pub/Sub for lightweight real-time notifications (cache invalidation broadcasts between service instances).

5.2 Why Not Memcached?

Feature	Redis	Memcached
Data structures	Strings, hashes, lists, sets, sorted sets, streams	Strings only
Persistence	RDB snapshots + AOF log	None (pure cache)
Distributed locking	Native (`SET NX EX`)	Requires external library
Cluster mode	Built-in (hash slots)	Client-side sharding
TTL granularity	Per-key, millisecond precision	Per-key, second precision
Pub/Sub	Built-in	Not available
Lua scripting	Built-in (atomic multi-step operations)	Not available

Redis's data structures (sorted sets for rate limiting, hashes for sessions, lists for recent txns) and distributed locking make it essential. Memcached is a simpler cache but lacks the primitives we need for locking, rate limiting, and session management.

5.3 Why Not Hazelcast or Apache Ignite?

- **Operational simplicity:** Redis is widely understood, has mature client libraries in every language, and AWS ElastiCache provides managed deployment.
- **Protocol compatibility:** The Redis protocol is a de facto standard. Tools like `redis-cli`, `RedisInsight`, and Prometheus exporters work out of the box.
- **Cost:** Hazelcast/Ignite require more memory per node (JVM overhead) and more operational expertise.
- Hazelcast/Ignite shine in compute-grid scenarios (distributed processing). We don't need that — we need a fast cache and lock manager.

5.4 Redis Deployment

```
Redis Cluster (3 masters + 3 replicas, multi-AZ)

┌──────────┐     ┌──────────┐     ┌──────────┐
│Master 1 │     │Master 2 │     │Master 3 │
│(AZ-1)   │     │(AZ-2)   │     │(AZ-3)   │
│Slots     │     │Slots     │     │Slots     │
│0-5460   │     │5461-10922│    │10923-16383│
└────┬─────┘     └────┬─────┘     └────┬─────┘
     │               │               │
     ▼               ▼               ▼
┌──────────┐     ┌──────────┐     ┌──────────┐
│Replica 1│     │Replica 2│     │Replica 3│
│(AZ-2)   │     │(AZ-3)   │     │(AZ-1)   │
└──────────┘     └──────────┘     └──────────┘

Each replica is in a different AZ from its master.
If AZ-1 goes down: Replica 2 and Replica 3 promote.
```

---

6. Elasticsearch — Search & Analytics

6.1 Why Elasticsearch for Search?

Covered in detail in the **Search Engine Choices** document. Summary:

Requirement	How ES Meets It
Full-text search	Inverted index with analyzers, tokenizers, stemmers
Fuzzy matching	Levenshtein distance, n-gram analysis
Real-time indexing	Near real-time (NRT) — documents searchable within 1 second of indexing
Aggregations	Category breakdowns, amount distributions, daily trends
Relevance scoring	TF-IDF / BM25 with custom boosting and function scores
Horizontal scaling	Add shards and nodes as data grows

6.2 ES is NOT a Primary Store

Elasticsearch is a **secondary index**, not a system of record. If ES loses data, we re-index from PostgreSQL. If ES goes down, we fall back to PostgreSQL's full-text search (degraded but functional). No financial data relies solely on ES.

7. Alternatives Evaluated & Rejected

7.1 CockroachDB / YugabyteDB (Distributed SQL)

Evaluated For	Verdict	Reasoning
Replacing PostgreSQL	Rejected	CockroachDB provides horizontal write scaling and geo-distribution, but our banking app runs in a single region (India — RBI data localization). Vertical PostgreSQL with read replicas handles our throughput (10K txns/sec). The added complexity of distributed consensus (Raft) increases latency by 2-5ms per write and introduces operational complexity we don't need at this scale.

7.2 Apache Cassandra

Evaluated For	Verdict	Reasoning
Audit log storage	Rejected in favor of MongoDB	Cassandra excels at write-heavy workloads but requires careful data modeling (denormalization, partition key design). MongoDB's document model is more natural for audit events with varying schemas. Cassandra's lack of ad-hoc query flexibility (no secondary indexes on arbitrary fields) makes compliance queries harder. MongoDB's aggregation pipeline is more powerful for the queries compliance officers run.

7.3 Amazon DynamoDB

Evaluated For	Verdict	Reasoning
Session / cache store	Rejected in favor of Redis	DynamoDB is a managed key-value store but has 1-5ms latency vs Redis's < 1ms. For OTP validation and session checks on every request, that difference matters at scale. DynamoDB also lacks Redis's distributed locking primitives (`SET NX EX`), Lua scripting, and Pub/Sub.

7.4 Apache Solr

Evaluated For	Verdict	Reasoning
Search engine	Rejected in favor of Elasticsearch	Both use Lucene under the hood. Elasticsearch has better REST API ergonomics, richer ecosystem (Kibana, Beats, Logstash), native Kubernetes operators, and larger community. Solr's ZooKeeper dependency adds operational overhead. ES is the industry standard for application search.

7.5 TimescaleDB

Evaluated For	Verdict	Reasoning
Time-series transaction data	Considered but deferred	TimescaleDB (PostgreSQL extension) excels at time-series workloads. Our transaction table is time-partitioned, which overlaps with Timescale's strengths. However, native PostgreSQL range partitioning + `pg_partman` achieves the same partition pruning without the extension dependency. Timescale may be reconsidered if we add real-time analytics dashboards with high-frequency queries.

7.6 Summary Decision Matrix

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│    Workload                        Winner         Runner-Up          │
│    ────────                       ───────        ──────────         │
│    Financial transactions          PostgreSQL     CockroachDB        │
│    Audit/event logs               MongoDB        Cassandra          │
│    Cache + sessions + locks        Redis          DynamoDB           │
│    Full-text search               Elasticsearch  Apache Solr        │
│    Time-series analytics          PostgreSQL*     TimescaleDB        │
│                                                                      │
│    * with native range partitioning                                  │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

---

8. Database-Per-Service Ownership Model

Each microservice owns its database. No service reads another service's tables directly. This is critical for microservice independence.

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│   Auth Service   │      │  Account Service  │      │ Transaction Svc  │
│                  │      │                   │      │                  │
│   ┌──────────┐   │      │   ┌──────────┐    │      │   ┌──────────┐   │
│   │ auth_db  │   │      │   │account_db│    │      │   │  txn_db  │   │
│   │          │   │      │   │          │    │      │   │          │   │
│   │ • users  │   │      │   │• accounts│    │      │   │• transact│   │
│   │ • roles  │   │      │   │• kyc_docs│    │      │   │• saga_st │   │
│   │ • perms  │   │      │   │• nominees│    │      │   │• idempot │   │
│   │ • mfa    │   │      │   │• mandates│    │      │   │          │   │
│   └──────────┘   │      │   └──────────┘    │      │   └──────────┘   │
└──────────────────┘      └──────────────────┘      └──────────────────┘
```

8.1 Cross-Service Data Access

Need	Mechanism
Transaction Service needs account balance	gRPC call to Account Service (synchronous)
Notification Service needs user email	Included in Kafka event payload (denormalized)
Reporting Service needs all data	Reads from PostgreSQL read replicas (CQRS pattern, materialized views)
Admin Service searches users	API call to Auth Service

8.2 Why Not a Shared Database?

Shared DB Problem	Impact
Schema coupling	Changing the accounts table breaks Transaction Service deployment
Deployment coupling	Can't deploy Account Service independently if Txn Service holds a lock
Scaling coupling	Can't scale transaction reads without scaling account writes
Team autonomy	Two teams can't own the same schema without coordination overhead

---

9. Data Consistency Across Stores

With four different data stores, consistency is a key challenge.

9.1 Consistency Model

Data Flow	Pattern	Consistency	Acceptable Staleness
PG → Redis (balance cache)	Event-driven invalidation via Kafka	Eventual	≤ 60 seconds (TTL fallback)
PG → ES (search index)	Kafka consumer (async)	Eventual	≤ 5 seconds (target)
PG → MongoDB (audit)	Kafka consumer (async)	Eventual	≤ 10 seconds
Redis → PG (session creation)	Write-through	Strong	0 (synchronous)

9.2 Source of Truth

```
PostgreSQL is the source of truth for ALL financial data.

If Redis disagrees with PG → PG wins (cache is stale, evict it).
If ES disagrees with PG → PG wins (re-index from PG).
If MongoDB audit is missing events → replay from Kafka (30-day retention).
```

---

10. Operational Considerations

10.1 Backup & Recovery

Store	Backup Strategy	RPO	RTO
PostgreSQL	Continuous WAL archiving + daily snapshots	< 1 min	< 15 min
MongoDB	Atlas continuous backup	< 5 min	< 30 min
Redis	RDB every 6h + AOF persistence	< 6 hours (acceptable — cache is rebuilt)	< 5 min (cluster failover)
Elasticsearch	Snapshot to S3 daily	< 24 hours (acceptable — re-index from PG)	< 1 hour

10.2 Monitoring

Store	Key Metrics	Alert Threshold
PostgreSQL	Replication lag, connection pool utilization, slow queries, VACUUM age	Repl lag > 10s, pool > 80%, query > 5s
MongoDB	Oplog window, replica set lag, WiredTiger cache utilization	Oplog < 24h, cache > 80%
Redis	Memory usage, eviction rate, connected clients, keyspace hit rate	Memory > 80%, evictions > 0 in production, hit rate < 90%
Elasticsearch	Cluster health, JVM heap, search latency, indexing lag	Health != GREEN, heap > 75%, search p99 > 500ms

---

11. Cost Analysis

11.1 Estimated Monthly Cost (Production)

Store	Instance Type	Configuration	Estimated Monthly Cost
PostgreSQL (RDS)	db.r6g.2xlarge	Primary + 1 standby + 2 read replicas	~$3,200
MongoDB Atlas	M40	3-node replica set	~$1,800
Redis (ElastiCache)	cache.r6g.xlarge	3 master + 3 replica cluster	~$2,400
Elasticsearch	r6g.xlarge.search	3 data + 2 master nodes	~$2,800
Total			**~$10,200/month**

11.2 Cost Justification

Alternative (single database)	Problem
Everything in PostgreSQL	Search is degraded (no fuzzy, no relevance scoring). Audit writes compete with transactional reads. No sub-ms caching. Estimated cost to scale PG to handle all workloads: ~$8,000/month (larger instances) + significant performance compromises.
Everything in MongoDB	No ACID transactions for financial operations. No foreign key integrity. No CHECK constraints for balance validation. Regulatory risk is unacceptable.

The polyglot approach costs ~$10K/month but delivers sub-ms caching, sub-second search, strong transactional guarantees, and immutable audit logs — each purpose-built for its workload.

End of Document — Database Choices v1.0